

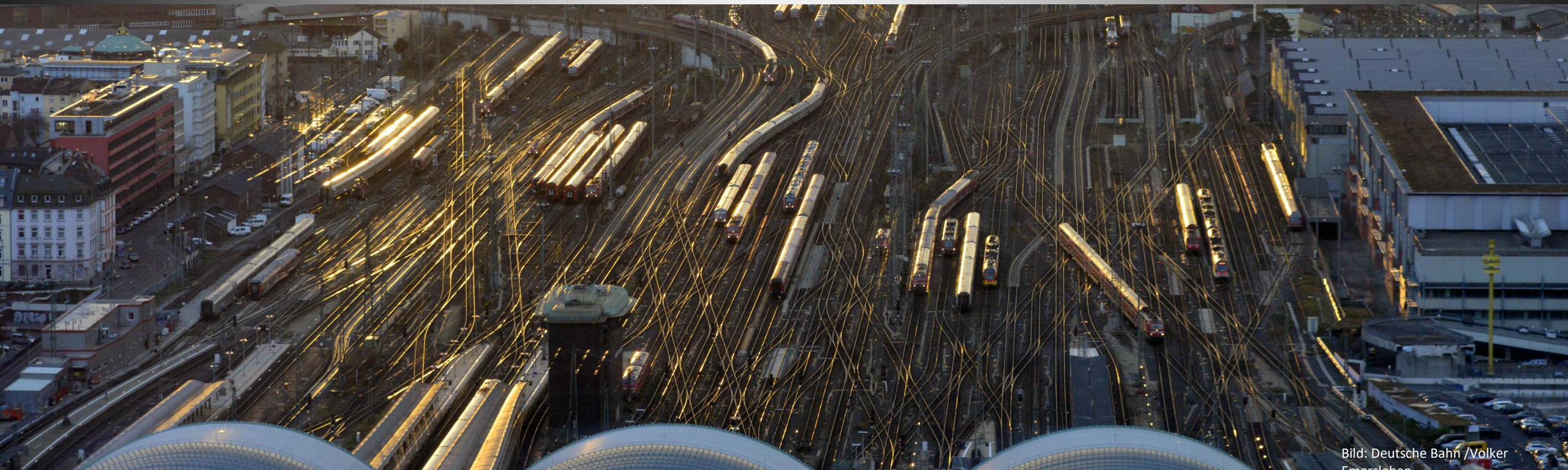


Bild: Deutsche Bahn /Volker
Emersleben

FINGERPRINTING MODERN C2 IMPLANTS THROUGH RUNTIME TELEMETRY

DOMINIK PHILLIPS, SEBASTIAN FELDMANN | 11.06.2026

WHO ARE WE?

- Dominik Phillips
- Sebastian Feldmann
- Members of the CSIRT of Deutsche Bahn AG

SCOPE OF THE TALK

- We want to fingerprint C2 implants at runtime with a reasonable performance and high confidence.
- Disclaimer: All demonstrated detections are not C2-specific.

HOW WE DETECT TODAY

- Blue teams use data from security sensors to write detection rules
- These rules aim to detect operator activity through sensor telemetry
- This is reactive: we only alert once the operator *does something*.
 - These rules rarely detect the implant itself!
 - Only an AV itself can do that
... and that's a black box we don't control

Action

```
[server] sliver (DEPENDENT_BROILER) > shell  
This action is bad OPSEC, are you an adult?  
Yes No  
←/→ toggle · enter submit · y Yes · n No
```

```
[*] Shell management: `shell ls`, `shell attach <id>`  
[*] Escape: press Ctrl-] to return to the Sliver client  
[*] Opening shell tunnel ...  
  
[*] Started remote shell [4] with pid 10672  
  
PS C:\Users\hackerman\Desktop> whoami  
hackercomputer\hackerman
```

Reaction

```
DeviceProcessEvents  
| where FileName =~ "cmd.Exe"  
| where ProcessCommandLine =~ "cmd.Exe /c whoami"  
| where InitiatingProcessFileName =~ "rundll32.Exe"
```

THE BLIND SPOT

What security-sensors share:

- Create Process
- Network connect
- File I/O
- Registry I/O
- Name Pipe
- Handles
- ...

These artifacts are used by Blue Teams to detect the operator

What security-sensors won't share is internally collected artifacts about:

- R(W)X allocations
- Stomped modules
- Hooked system calls / ntdll trampolines
- Timer/APC abuse (sleep obfuscation, threadless injection)
- Abnormal threads
- ...

Why:

- Rich and complex data requiring per-process correlation
- High volume, expensive to monitor continuously
- Conclusions about detection capabilities and the characterization of these as intellectual property

These artifacts, combined with static and dynamic signatures, are used by AV to identify and characterize implants

We're entirely dependent on the operator making a mistake, or the AV flagging the implant for us.

IMPLANTS ARE DETECTABLE (1)

- In an Incident Response case, identifying the implant is often straightforward
 - Well-known scanners - Moneta, PE-Sieve or Hunt-Sleeping-Beacons – usually find them
 - They scan for memory-artifacts, suspicious callstacks ...
 - Results aren't always a definitive match, but consistently a strong indicator
- These tools usually work. But they require manual execution and have FPs

```
Moneta v1.0 | Forrest Orr | 2020

ekko.exe : 4356 : x64 : ekko.exe
0x00007FF79D370000:0x00029000 | EXE Image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | Header | 0x00029000 | Primary image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .text | 0x00029000 | Primary image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .rdata | 0x00029000 | Primary image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .data | 0x00029000 | Primary image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .pdata | 0x00029000 | Primary image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .fptable | 0x00029000 | Primary image
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .rsrc | 0x00029000 | Primary image base | Modified PE header | Inconsistent +x between disk and memory | Modified code
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
0x00007FF79D370000:0x00029000 | RWX | .reloc | 0x00029000 | Primary image base | Modified PE header | Inconsistent +x between disk and memory | Modified code
y | Modified code | Inconsistent +x between disk and memory | Modified code | Inconsi
Thread 0x00007FF79D371AF0 [IID 0x0000410]

... scan completed (0.125000 second duration)
```

```
Hunt-Sleeping-Beacons | @thefLinkk

* Detections for: ekko.exe (4356)
! Thread 1040 | Blocking Timer detected | Thread's blocking state triggered by ntdll!RtlpTpTimerCallback | Severity: HIGH
! Suspicious Timer | A suspicious timer callback was identified pointing to ntdll!NtContinue | Severity: CRITICAL
! Suspicious Timer | A suspicious timer callback was identified pointing to ntdll!NtContinue | Severity: CRITICAL
! Thread 1040 | Abnormal Page in Callstack | Callstack contains private memory regions | Severity: MID
! Thread 1040 | Module Stomping | Callstack contains stomped module: ekko | Severity: LOW
! Thread 1040 | Non-Executable Page in Callstack | Callstack contains memory regions marked as non-executable | Severity: MID
! Thread 7564 | Blocking Timer detected | Thread's blocking state triggered by ntdll!RtlpTpTimerCallback | Severity: HIGH
! Thread 7564 | Abnormal Intermodular Call | ntdll!TpAllocTimer called KERNELBASE!WaitForSingleObjectEx. This indicates module-pro

* Scanned: 1 processes and 2 threads in 0.063 seconds
```


IMPLANTS ARE DETECTABLE (2)

- What Blue Teams typically do is embed these scanners into automated live-response procedures.
 - This leads to implant detection after operator detection.
- But why this reactive approach?
 - The implant was never truly hidden - the artifacts were always present, only hidden from the detection pipeline.
- To move away from this trigger-based model, we need to integrate scanner signals live and per-process at runtime.
- The goal is to know: this process might be an implant based on observed runtime artifacts. This enables:
 - Fine-grained operator-detection rules
 - In the best case, implant detection before operator detection occurs

SYSTEMIZING THE IMPLANT – CAPABILITIES

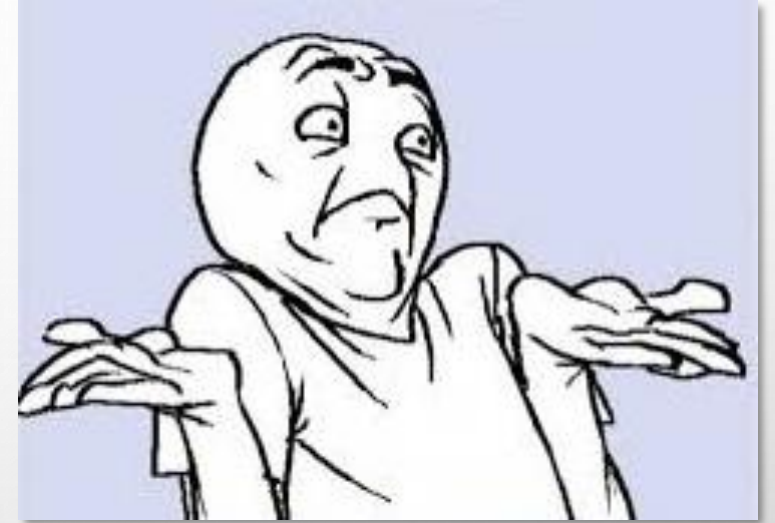
- We structure implant capabilities in two peer axes:
 - Primitives: functional capabilities the implant provides
 - Evasion: techniques to conceal primitives
- Every implant needs a core set of primitives to function:
 - ***Command & Control Channel***
 - ***In-Memory Execution***
 - ***This is the core set of capabilities that, in our view, defines an implant.***
 - Everything else is mission-driven and varies per engagement

Primitives		Evasion (stealth)
Core (every implant)	Scope-Specific (mission-driven)	■ Dynamic code ■ Sleep masking ■ API redirection ■ Callstack spoofing ■ ...
■ C2 channel ■ Network-Tunneling ■ In-Memory-Module exec (e.g.BOFs) ■ ...	■ Credential dumping ■ File / registry ■ Process injection ■ Lateral movement ■ ...	

- Evasion is not optional but part of the implant workflow. However, evasion techniques leave their own artifacts.
- Only the scope specific primitives are usually covered by the blue-team

DETECTING EVASION WITH ETW

- Tracing via ETW
 - Every process leaves traces at the resource level. On Windows, the most common and minimally invasive way to observe this is ETW (Event Tracing for Windows).
- From Point-in-Time Scans to Continuous Telemetry
 - What if we could do what Moneta, PE-Sieve, and HSB detect - but continuously, through tracing instead of snapshots?
 - Memory artifacts
 - Call Stacks anomalies
 - Timer / APC activity
 - The implant still relies on system resources to achieve evasion
 - Sleep masking requires timers → timers rely on the thread pool → thread pool activity is traceable
 - Dynamic code requires allocations or manipulation of existing memory → related page faults are traceable
- ***Implant's core primitives + evasion telemetry, aggregated per process, correlated over time - that's what we refer to as the implant fingerprint***



WHY IS PURE OBSERVATION HARD?

- There is no evasion ETW-Provider
 - There's no ETW event that says 'timer-based sleep obfuscation detected.' There's a threadpool timer creation event. The gap between those two is what we have to bridge.
 - A passive observer only sees what the OS surfaces - events built for system diagnostics, not security
- ***The challenge: pinpoint an evasion technique using ETW events that weren't built to describe it***



EXAMPLE (EVASION): QUEUE WORK TO THREADPOOL (1)

- Modern C2 payloads spend a significant amount of time at rest. During these periods, they seek to minimize their footprint to evade detection
 - Encrypt, pack or compress regions while at rest
 - If the implant just encrypts itself but keeps its own threads parked, the threads return addresses still point into a region that can be scanned
- So, they use a timer-driven asynchronous callback approach - built on the Windows thread pool (CreateThreadPoolTimer, SetThreadpoolTimer, ...)
 - With a timer (a due time) e.g., to enable sleep-masking
 - Or, with an immediate work item e.g., To conceal the call stack during module load (module proxying)
- *Every time a work item (due timer or immediate work item) is queued to the thread pool - the OS must do its work: enqueue it onto the pool ...*
 - ***This work is not optional***



EXAMPLE (EVASION): QUEUE WORK TO THREADPOOL (2)

```
[Guid("{c861d0e2-a2c1-4d36-9f9c-970bab943a12}")]  
class ThreadPool : EventTrace { };
```

```
uint32 SubProcessTag; [EventType{34}]  
class TP_V2_CBEnqueue : ThreadPool {  
    uint32 PoolId;  
    uint32 TaskId;  
    uint32 CallbackFunction;  
    uint32 CallbackContext;  
};
```

- Timers are based on the windows thread pool
- Monitor CallbackFunction of **EventType{34}** events through **TP_V2_CBEnqueue**
- **CallbackFunction**: the callback function to which execution resumes once the timer has expired
 - According to our observations, a timer callback - for instance, to ntdll!Ntcontinue or kernel32!Loadlibrary - most likely indicates sleepmasking or module proxying
 - A timer callback to dynamic code regions also is a strong IOC

EXAMPLE (EVASION): QUEUE WORK TO THREADPOOL (3)

- To control how execution flows or is resumed, wrapper functions are often used;

- NtContinue can restore a thread's execution context
- But similar wrapper patterns can exist in higher-level apis

```
Ekko / Src / Ekko.c
Code Blame 112 lines (88 loc) · 4.06 KB
44
45 if ( CreateTimerQueueTimer( &hNewTimer, hTimerQueue, RtlCaptureContext, &CtxThread, 0, 0, WT_EXECUTEINTIMERTHREAD ) )
46 {
```

```
97 CreateTimerQueueTimer( &hNewTimer, hTimerQueue, NtContinue, &RopProtRW, 100, 0, WT_EXECUTEINTIMERTHREAD );
98 CreateTimerQueueTimer( &hNewTimer, hTimerQueue, NtContinue, &RopMemEnc, 200, 0, WT_EXECUTEINTIMERTHREAD );
99 CreateTimerQueueTimer( &hNewTimer, hTimerQueue, NtContinue, &RopDel, 300, 0, WT_EXECUTEINTIMERTHREAD );
100 CreateTimerQueueTimer( &hNewTimer, hTimerQueue, NtContinue, &RopExec, 400, 0, WT_EXECUTEINTIMERTHREAD );
101 CreateTimerQueueTimer( &hNewTimer, hTimerQueue, NtContinue, &RopExec, 500, 0, WT_EXECUTEINTIMERTHREAD );
102 CreateTimerQueueTimer( &hNewTimer, hTimerQueue, NtContinue, &RopExec, 600, 0, WT_EXECUTEINTIMERTHREAD );
```

```
Cronos / src / Cronos.c
Code Blame 148 lines (120 loc) · 5.15 KB
56 Image.Length = Image.MaximumLength = ImageSize;
57
58 // Creating the waitable timers.
59 hProtectionRWTimer = CreateWaitableTimerW(NULL, TRUE, L"ProtectionRWTimer");
```

```
127
128 // Setting the timers.
129 if (!SetWaitableTimer(hDecryptionTimer, &decryptionDueTime, 0, NtContinue, &ctxDecryption, FALSE) ||
130 !SetWaitableTimer(hProtectionRWTimer, &protectionRWDueTime, 0, NtContinue, &ctxProtectionRW, FALSE) ||
131 !SetWaitableTimer(hProtectionRWTimer, &protectionRWDueTime, 0, NtContinue, &ctxProtectionRW, FALSE) ||
132 !SetWaitableTimer(hEncryptionTimer, &encryptionDueTime, 0, NtContinue, &ctxEncryption, FALSE))
```

```
* Process: ekko.exe queued a suspicious timer on: ZwContinue
* Process: ekko.exe queued a suspicious timer on: ZwContinue
* Process: ekko.exe queued a suspicious timer on: ZwContinue
* Process: ekko.exe queued a suspicious timer on: RtlCaptureContext
* Process: ekko.exe queued a suspicious timer on: ZwContinue
* Process: ekko.exe queued a suspicious timer on: ZwContinue
* Process: ekko.exe queued a suspicious timer on: ZwContinue
```

```
misc / bypasses / WorkItemLoadLibrary.c
Code Blame 81 lines (65 loc) · 2.19 KB
48 if (swtch)
49 {
50     IMPORTAPI(L"NTDLL.dll", RtlQueueWorkItem, NTSTATUS, PVOID, PVOID, ULONG);
51
52     if (NT_SUCCESS(RtlQueueWorkItem(&LoadLibraryW, (PVOID)libraryName, WT_EXECUTEDEFAULT)))
53     {
```

- To evade callstack-analysis, some function-calls are proxied through threadpools, such as:

- LoadLibraryA
- NtReadVirtualMemory
- ...

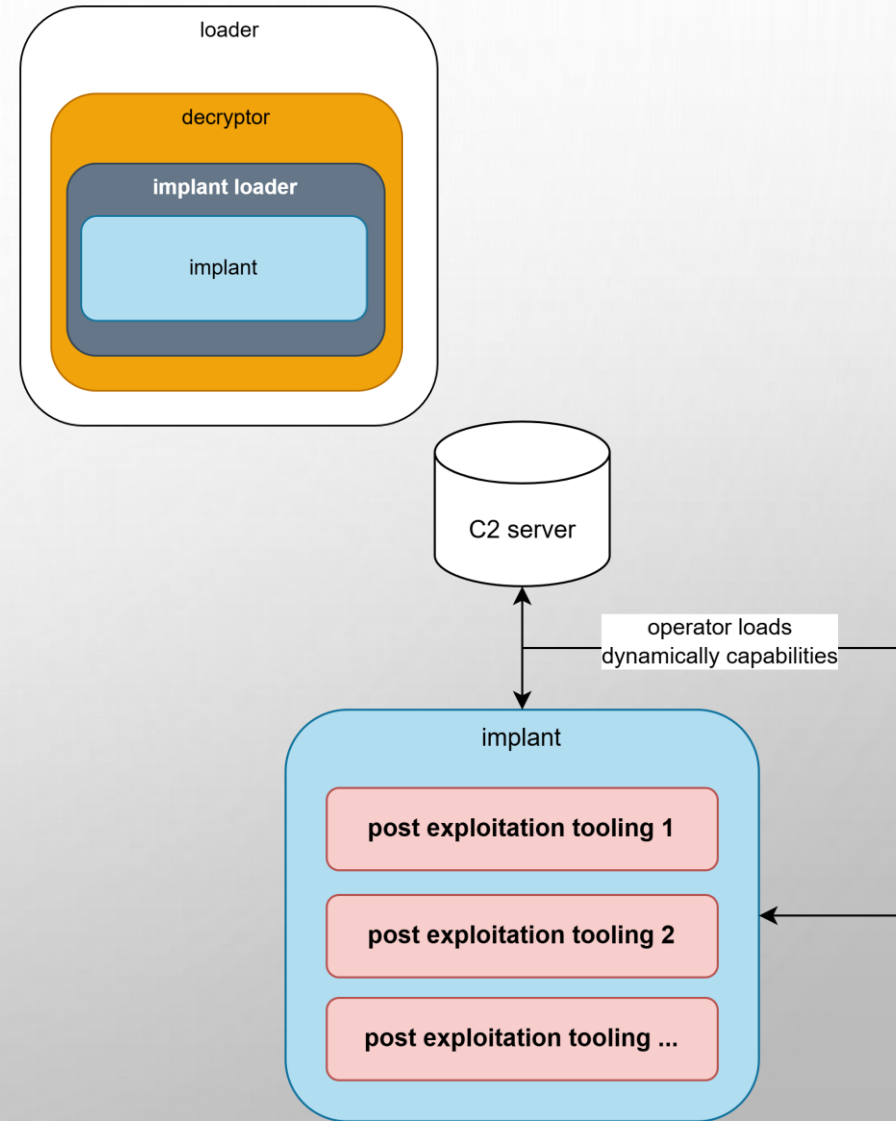
```
* Process: workitem-loadlibrary.exe queued a suspicious timer on: LoadLibraryA
```

DEMO:
TIMER BASED SLEEPMASK



EXAMPLE (EVASION): ALLOCATED MEMORY (1)

- Modern C2 payloads arrive as loaders that decrypt, decompress, or reconstruct the real implant only at runtime, in memory
- Once unpacked, the implant is rarely "finished" - operators load dynamically new capabilities mid-engagement: post-exploitation tooling



EXAMPLE (EVASION): ALLOCATED MEMORY (2)

- To execute, code must be resident in a process address space and executable at some point.
- Every time code is brought in - the OS must do its work: allocations and page faults occur
 - *This work is not optional*

```
282  /* Handle the allocation and copying of the sections we're going to use
283  * for right now I'm just VirtualAlloc'ing memory, this can be changed to
284  * other methods, but leaving that up to the person implementing it. */
285  for (counter = 0; counter < coff_header_ptr->NumberOfSections; counter++) {
286      coff_sect_ptr = (coff_sect_t*)(coff_data + sizeof(coff_file_header_t) + (sizeof(coff_sect_t) * counter));
287      DEBUG_PRINT("Name: %s\n", coff_sect_ptr->Name);
288      DEBUG_PRINT("VirtualSize: 0x%X\n", coff_sect_ptr->VirtualSize);
289      DEBUG_PRINT("VirtualAddress: 0x%X\n", coff_sect_ptr->VirtualAddress);
290      DEBUG_PRINT("SizeOfRawData: 0x%X\n", coff_sect_ptr->SizeOfRawData);
291      DEBUG_PRINT("PointerToRelocations: 0x%X\n", coff_sect_ptr->PointerToRelocations);
292      DEBUG_PRINT("PointerToRawData: 0x%X\n", coff_sect_ptr->PointerToRawData);
293      DEBUG_PRINT("NumberOfRelocations: %d\n", coff_sect_ptr->NumberOfRelocations);
294      relocationCount += coff_sect_ptr->NumberOfRelocations;
295      /* NOTE: When changing the memory loading information of the loader,
296       * you'll want to use this field and the defines from the Section
297       * Flags table of Microsofts page, some defined in COFFLoader.h */
298      DEBUG_PRINT("Characteristics: %X\n", coff_sect_ptr->Characteristics);
299
300      #ifdef _WIN32
301          DEBUG_PRINT("Allocating 0x%x bytes\n", coff_sect_ptr->VirtualSize);
302          /* NOTE: Might want to allocate as PAGE_READWRITE and VirtualProtect
303           * before execution to either PAGE_READWRITE or PAGE_EXECUTE_READ
304           * depending on the Section Characteristics. Parse them all again
305           * before running and set the memory permissions. */
306          sectionMapping[counter] = VirtualAlloc(NULL, coff_sect_ptr->SizeOfRawData, MEM_COMMIT | MEM_RESERVE | MEM_TOP_DOWN, PAGE_EXECUTE_READWRITE);
307
308          #ifdef DEBUG
309              sectionSize[counter] = coff_sect_ptr->SizeOfRawData;
310          #endif
311          if (sectionMapping[counter] == NULL) {
312              DEBUG_PRINT("Failed to allocate memory\n");
313          }
314          DEBUG_PRINT("Allocated section %d at %p\n", counter, sectionMapping[counter]);
315          if (coff_sect_ptr->PointerToRawData != 0) {
316              memcpy(sectionMapping[counter], coff_data + coff_sect_ptr->PointerToRawData, coff_sect_ptr->SizeOfRawData);
317          }
318          else {
319              memset(sectionMapping[counter], 0, coff_sect_ptr->SizeOfRawData);
320          }
321      #endif
322      DEBUG_PRINT("Total Relocations: %d\n", relocationCount);
323      /* Allocate and setup the GOT for functions, same here as above. */
324      /* Actually allocate enough for worst case every relocation, may not be needed, but hey better safe than sorry */
325      #ifdef _WIN32
326          #ifdef _WIN64
327              functionMapping = (void **)VirtualAlloc(NULL, relocationCount*8, MEM_COMMIT | MEM_RESERVE | MEM_TOP_DOWN, PAGE_EXECUTE_READWRITE);
328          #else
329              functionMapping = (void **)VirtualAlloc(NULL, relocationCount*8, MEM_COMMIT | MEM_RESERVE | MEM_TOP_DOWN, PAGE_EXECUTE_READWRITE);
330          #endif
331      #endif
332  }
```

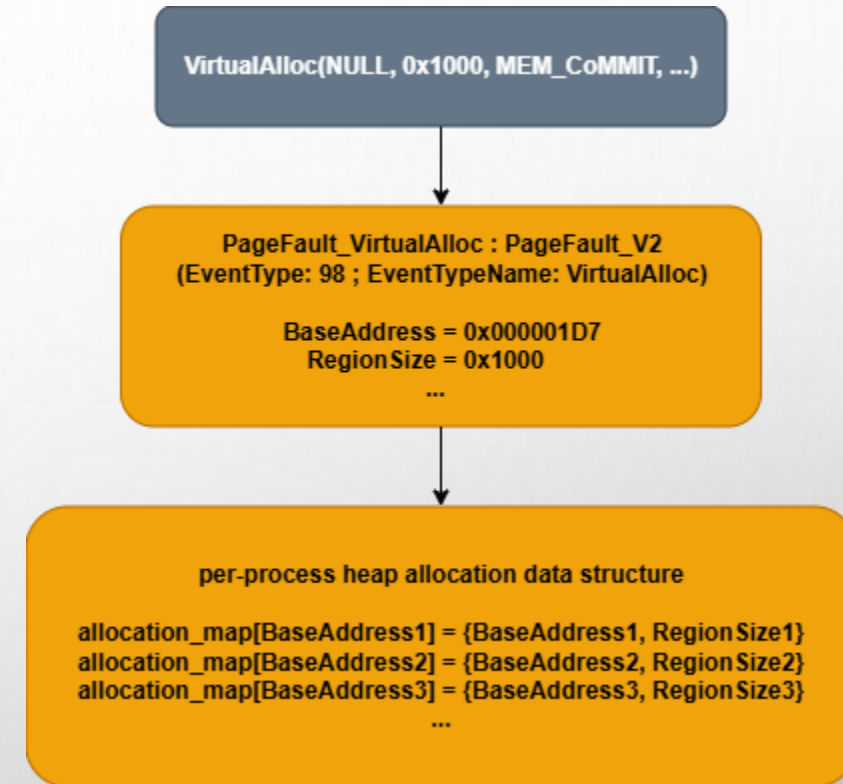
<https://github.com/trustedsec/COFFLoader/blob/main/COFFLoader.c#L305>

NAÏVE APPROACH: TRACK ALL ALLOCATIONS (1)

```
[Guid("{3d6fa8d3-fe05-11d0-9dda-00c04fd7ba7c}"), EventVersion(2)] class PageFault_V2 :  
MSNT_SystemTrace { };
```

```
[EventType{98, 99}, EventTypeNames{"VirtualAlloc", "VirtualFree"}] class PageFault_VirtualAlloc :  
PageFault_V2 {  
    uint32 BaseAddress;  
    object RegionSize;  
    uint32 ProcessId;  
    uint32 Flags;  
};
```

- Track all allocations using the PageFault_VirtualAlloc event and aggregate them into a map of dynamic memory regions.
- However, this provider is not designed for security purposes:
 - It does not expose memory protection flags
 - We can see that memory was allocated, but we lose key context about its intended role



NAÏVE APPROACH: TRACK ALL ALLOCATIONS (2)

```
[EventType{10, 11, 12, 13, 14, 15}, EventTypeName{"TransitionFault", "DemandZeroFault", "CopyOnWrite",  
"GuardPageFault", "HardPageFault", "AccessViolation"}]
```

```
class PageFault_TypeGroup1 : PageFault_V2
```

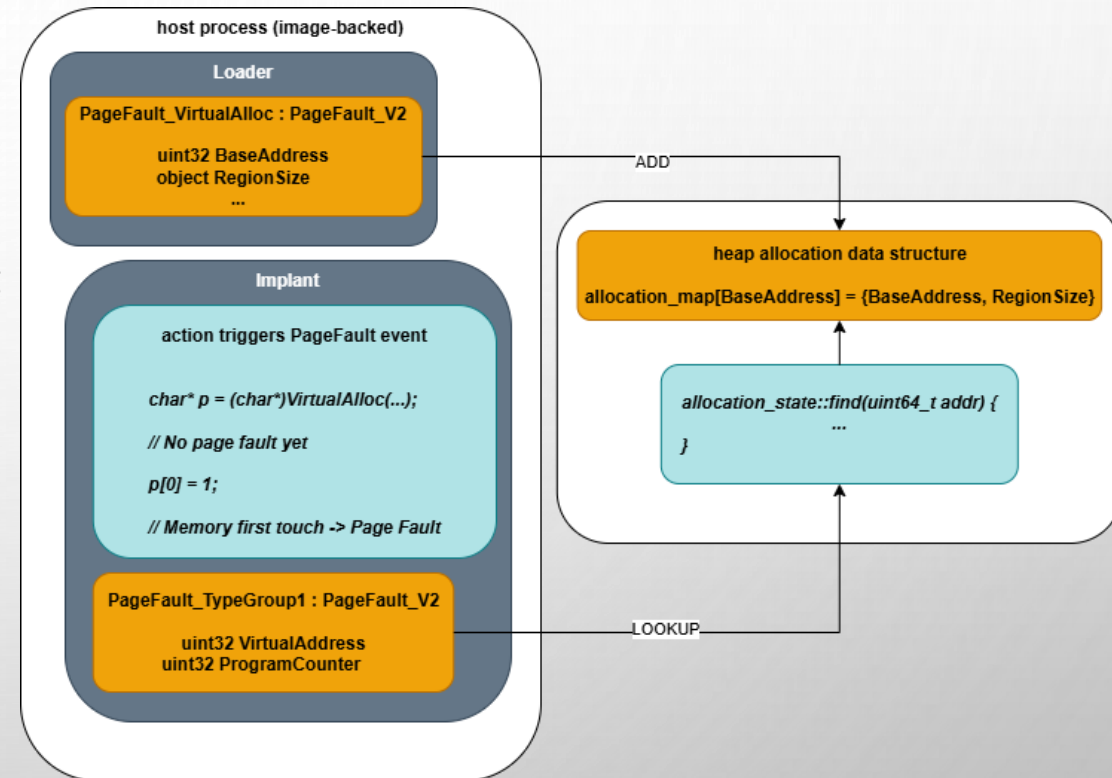
```
{
```

```
    uint32 VirtualAddress;
```

```
    uint32 ProgramCounter;
```

```
};
```

- Monitor events that contain an instruction pointer (IP) and correlate them with the dynamic allocation map. E.g. Pagefaults
- If the IP falls within a previously tracked dynamic allocation, the code is executing from dynamically allocated memory.
- Pros:
 - Detects dynamic code execution without performing memory scans
 - Straightforward correlation model: $IP \in \text{allocation map} \rightarrow \text{dynamic code execution}$
- Limitations
 - Performance cost: Every relevant event must be correlated against the allocation map, making the approach CPU-intensive at scale.
 - Module stomping: Missed because execution occurs within the address range of a legitimate image.
 - VirtualProtect-based techniques: Missed when memory originates from a legitimate image mapping and is only later re-protected to become executable.



OUR APPROACH (1)

- Trace Callstack of events that are frequently (but not too frequently) emitted, such as:
 - ObHandle -> whenever a process opens any kind of handle
 - Pagefault -> whenever memory is allocated, accessed first time, or paged in...
 - ...
- For example, monitor callstacks of **DemandZeroFault**, **CopyOnWrite** or **TransitionFault** events through **StackWalk_Event**
 - DemandZeroFault** : The process accesses a new memory page that has never been used before
 - CopyOnWrite**: A DLL was modified in memory
 - TransitionFault**: A page was removed from the working set of a process and is now requested again.

```
[Guid("{def2fe46-7bd6-4b80-bd94-f57fe20d0ce3}")]
```

```
class StackWalk : MSNT_SystemTrace {
```

```
[EventType{32}, EventTypeName{"Stack"}]
```

```
class StackWalk_Event : StackWalk
```

```
{
```

```
    uint64 EventTimeStamp;
```

```
    uint32 StackProcess;
```

```
    uint32 StackThread;
```

```
    uint32 Stack1;
```

```
    uint32 Stack192;
```

```
};
```

The screenshot displays a C++ code snippet for a stack walking function and a corresponding call stack dump. The code defines a function `on_stackwalk` that takes a handler context and a record, and returns a stack of frames. The stack is represented as a vector of `stackwalk_frame` objects. The code uses `reinterpret_cast` to convert the record's user data into a stack of frames. The stack is then processed to extract the process ID, thread ID, and the stack frames themselves. The stack frames are then used to generate a call stack dump.

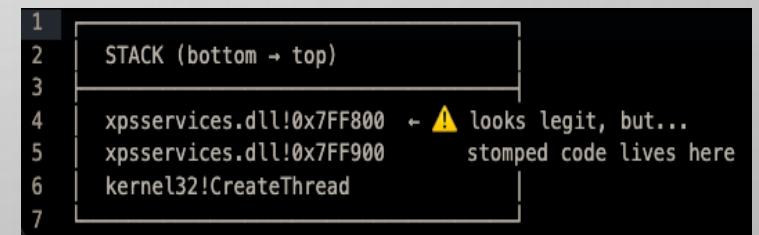
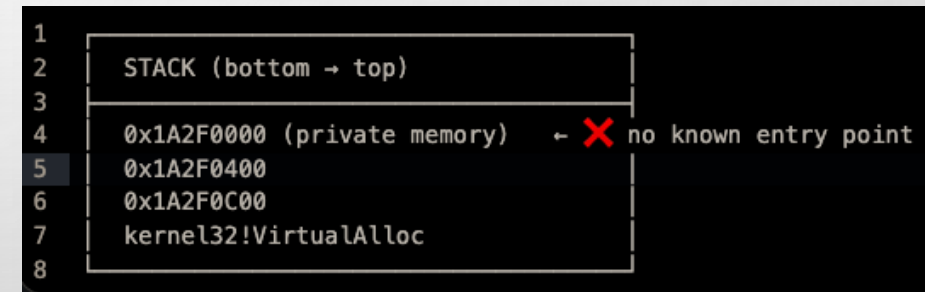
```
359 void on_stackwalk(handler_context& ctx,  
360 const EVENT_RECORD& record,  
361 const krabs::trace_context& context) noexcept {  
362     {  
363         static constexpr size_t stackwalk_frame_offset = sizeof(uint64_t) + 2 * sizeof(uint32_t);  
364         auto* base = static_cast<const uint8_t*>(record.UserData);  
365         auto pid = *reinterpret_cast<const uint32_t*>(base + sizeof(uint64_t));  
366         auto timestamp = *reinterpret_cast<const uint64_t*>(base);  
367         auto tid = *reinterpret_cast<const uint32_t*>(base + sizeof(uint64_t) + sizeof(uint32_t));  
368         auto* frames = reinterpret_cast<const uint64_t*>(base + stackwalk_frame_offset);  
369         auto n_frames = (record.UserDataLength - stackwalk_frame_offset) / sizeof(uint64_t);  
370         if (n_frames == 0 || is_kernel_addr(frames[0]))  
371             return;  
372         stackwalk_event e{  
373             .timestamp = timestamp,  
374             .pid = pid,  
375             .tid = tid,  
376             .frames = std::span<const uint64_t>(frames, n_frames)  
377         };  
378         auto proc = ctx.tree.lookup(pid);  
379         if (proc.has_value())  
380         {  
381             ctx.interpreter.on_stackwalk(*proc.value(), e);  
382         }  
383     }  
384 }
```

The call stack dump shows the following information:

Name	Value
> base	0x000002d5b27e6668 "6M6LF\1a"
> context	{schema_locator={cache_={size=0x0000000000000004 }}}
> ctx	{tree={mu_={Myhandle=0x0000000000000000 [...] } by_
> e	{timestamp=0x00001aa34cf54df3 pid=0x000016cc tid=0
> timestamp	0x00001aa34cf54df3
> pid	0x000016cc
> tid	0x00001b4c
> frames	{size=0x000000000000000f }
> [0x00000000]	0x00007ffaee1114
> [0x00000001]	0x00007ffaee1114
> [0x00000002]	0x00007ffaee1114
> [0x00000003]	0x00007ffaee1114
> [0x00000004]	0x00007ffaee1114
> [0x00000005]	0x00007ffaee1114
> [0x00000006]	0x00007ffaee1114
> [0x00000007]	0x00007ffaee1114
> [0x00000008]	0x00007ffaee1114
> [0x00000009]	0x00007ffaee1114
> [0x0000000a]	0x00007ffaee1114
> [0x0000000b]	0x00007ffaee1114
> [0x0000000c]	0x00007ffaee1114
> [0x0000000d]	0x00007ffaee1114
> [0x0000000e]	0x00007ffaee1114

OUR APPROACH (2)

- Callstack analysis allows efficiently identifying dynamic code execution
- Legitimate threads typically originate from `ntdll!RtlUserThreadStart`.
 - For legitimate execution:
 - The stack unwinds through known modules
 - Each frame can be resolved using registered unwind metadata (e.g., .pdata entries)
 - For dynamic or stomped code:
 - No valid unwind metadata exists for the executing region
 - Stack unwinding terminates early or produces frames that cannot be resolved to a loaded module



OUR APPROACH - LIMITATIONS

- Looking strictly at the condition, it is prone to false positives:
 - Incomplete/failed stack walks (missing unwind info, optimized tail calls)
 - Legitimate trampolines/thunks (some AV hooking engines, hot-patching)
- Dynamic code alone can be fully legit
 - JIT (.NET, V8, Java)
- This approach alone is too prone to FPs 😞

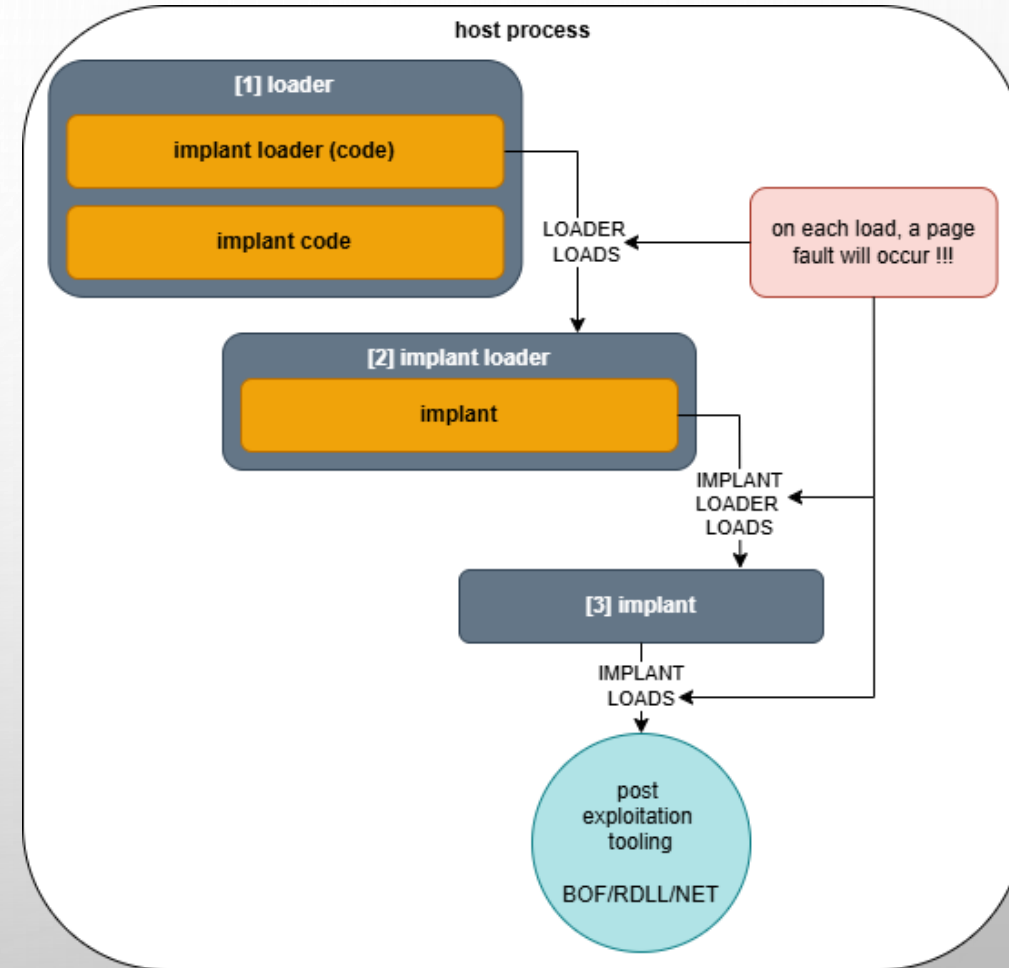


DEMO: DYNAMIC CODE



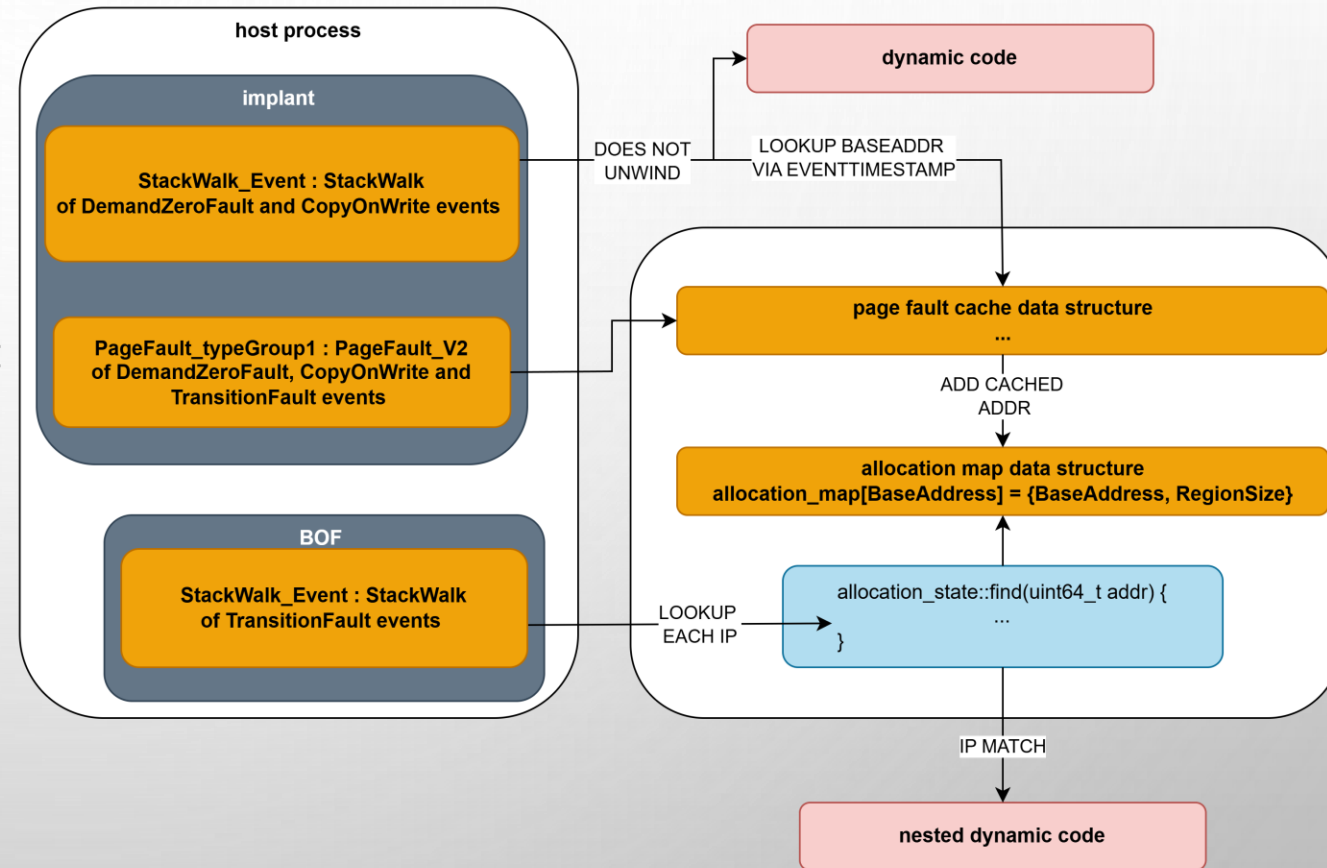
REDUCING NOISE: NESTED DYNAMIC CODE

- In practice, modern implants rarely unpack in a single step - we consistently observe a chain of dynamic code generations.
- One executable region spawns, writes, and transfers execution into another. **We call this nested dynamic code**
 - This happens either as part of multi-staged unpacking/initialization, or when an implant executes a post-exploitation module such as a BOF.
- Nested dynamic code is common in implant workflows but appears rare in legitimate software.
 - JIT engines allocate executable memory, but the allocator itself lives in a signed, module-backed image
 - Legitimate: module-backed code → dynamic code (one layer)
 - Implant: dynamic code → dynamic code (n layers)
- **During each unpacking/initialization-step page-faults will occur!**



TRACKING NESTED DYNAMIC CODE [1]

1. Cache the BaseAddress of **DemandZeroFault**, **CopyOnWrite** and **TransitionFault** events through **PageFault_TypeGroup1**
2. Monitor callstacks of **DemandZeroFault**, **CopyOnWrite** events through **StackWalk_Event**
 - If callstack does not unwind we consider the executing code as dynamic code = **Tag** process **dynamic code**
 - Furthermore, a broken callstack of a pagefault-event reveals dynamic code allocating pages
 - Use the EventTimeStamp to look up the cached BaseAddress associated with the corresponding PageFault_TypeGroup1 event
 - Store the BaseAddress in the allocation map



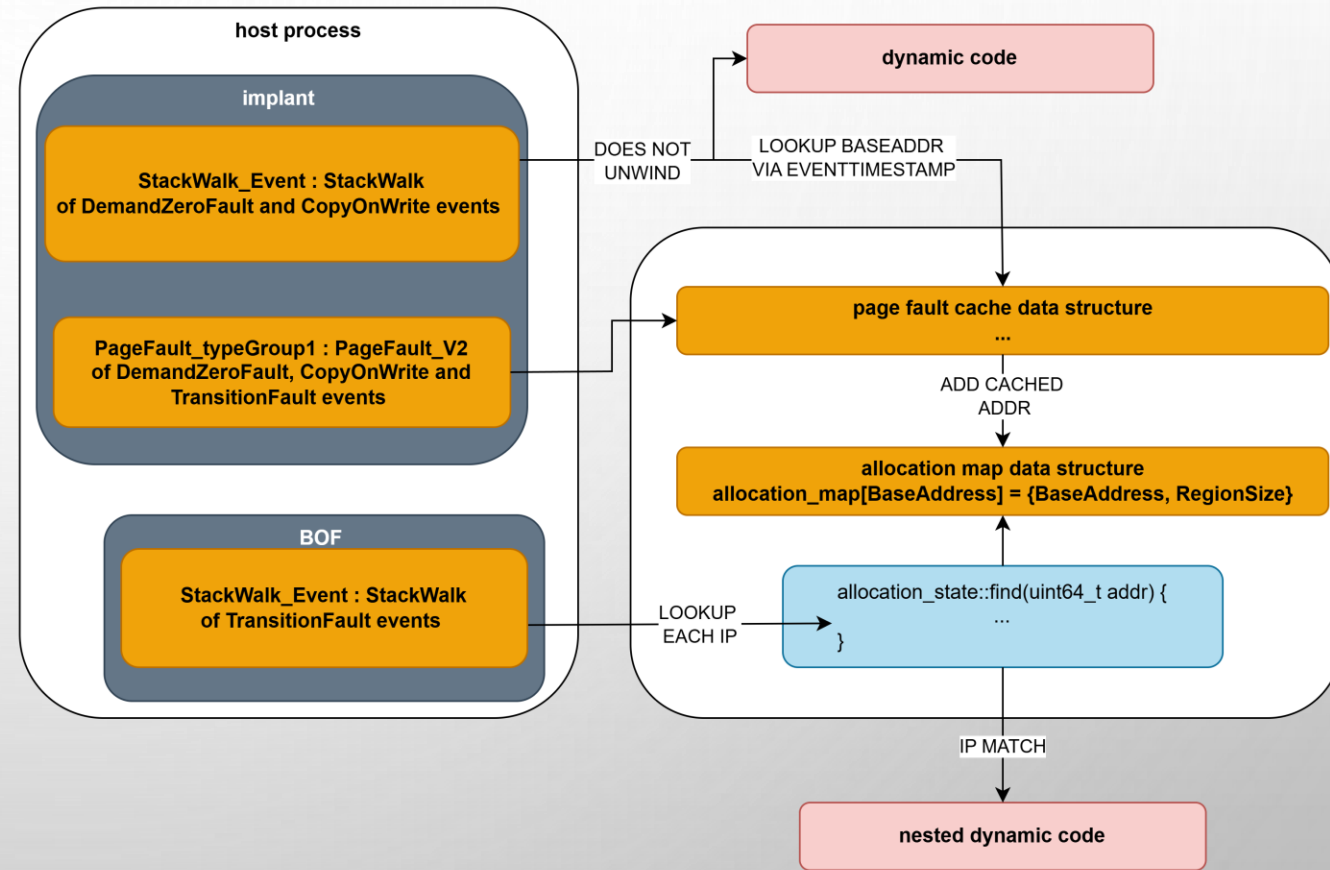
TRACKING NESTED DYNAMIC CODE [2]

3. To characterize the stored page in step 2 as nested dynamic code, we need a IP that points to that page

- We monitor the Stackwalk-Events of Transition-Fault events to collect PCs

4. Cross-reference callstacks (i.e. each IP) of **TransitionFault** events through **StackWalk_Event** with the allocation map populated in step 2

- If the allocator of a dynamic code region is itself dynamic code, we have a stronger IOC = **Tag** process **nested dynamic code**



DEMO:
NESTED DYNAMIC CODE



TRACKING NESTED DYNAMIC CODE [3]

- Dynamic code is a known - but weak - IOC
 - Too broad, too many false positives.
- Nested dynamic code is a strong IOC:
 - It precisely fingerprints a key behavioral primitive of implants and loaders.
- Tracking it via page-fault telemetry is efficient and low-noise.
- An implant cannot evade page-faults themselves - this is simply how memory works in windows.
 - You cannot unpack without them.
- We get this for free via ETW
 - No hooks, no patches - just the OS reporting its own internals.

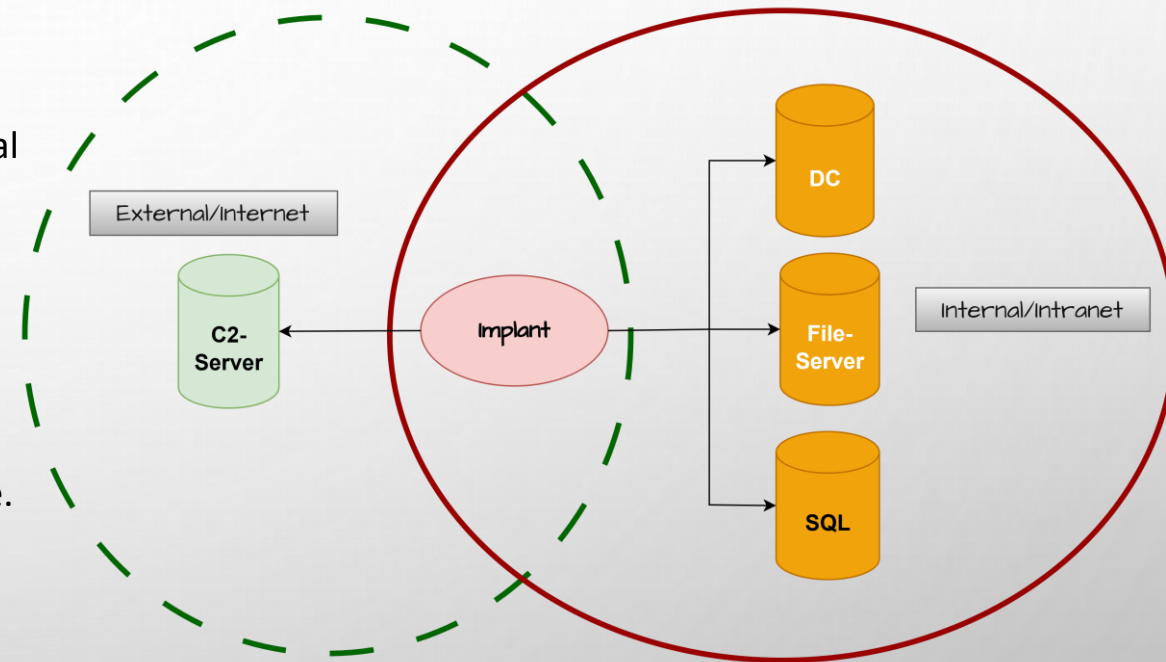
REDUCING FALSE POSITIVES: A STRONGER CASE

- All previously presented approaches can introduce false positives, because in practice there are legitimate processes that can perform similar actions
- To build a stronger, lower false-positive result, we must move to a multi-signal composition:
 - **A multi-signal composition of core primitives and evasion telemetry, aggregated per process and correlated over time**
 - **This is what we define as the implant fingerprint.**



EXAMPLE (CORE PRIMITIVE): SUSPICIOUS NETWORK TUNNELING ACTIVITY

- An implant must communicate with its external endpoint - C2-Server
- An implant must communicate with an internal service
 - e.g. network-reconnaissance/lateral-movement
- A process connecting to an external endpoint (C2-Server) and multiple internal services (DC, SMB-Share, ...) is rare
 - Kerberos
 - Ldap
 - Smb
 - Rdp
 - ...
- Aggregate per-process the internal services and external endpoints, over time.
Tag the process when two conditions hold:
 - It has opened a suspicious combination of internal services
 - It is also talking to an external endpoint
- `[Guid("{9a280ac0-c8e0-11d1-84e2-00c04fb998a2}"), EventVersion(2)] class TcpIp : MSNT_SystemTrace { };`
- `[EventType{10}, EventTypeName{"SendIPV4"}] class TcpIp_SendIPV4 : TcpIp { uint32 daddr; uint32 dport; ...};`



EXAMPLE (CORE PRIMITIVE): SUSPICIOUS COMBINATION OF ADMINISTRATIVE DLLS

```
/tmp/CS-Situational-Awareness-BOF$ grep -rhoP '[A-Za-z0-9_]+(?:=\$)' --include="*.c" --include="*.h" . | sort -u
```

- Most bofs provide administrative functionality that are, for example, implemented in the win32 API:
 - Netenum
 - Adcs_enum
 - Ldapsearch
 - ...
- The more bofs an implant executes, the more dlls are loaded over time
 - Mostly dlls which only administrative tools should load
- Aggregate loaded dlls over time. **Tag** the process if a certain suspicious combination is reached
 - [Guid("{2cb15d1d-5fc1-11d2-abe1-00a0c911f518}"), eventversion(2)] class image : msnt_systemtrace { };
 - [eventtype(10), eventtypename("load")] class image_v1_load : image_v1 { uint32 imagebase; uint32 imagesize; uint32 processid; string filename; };

mimikatz.exe	
Module Name	Imports
szAnsi	(nFunctions)
ADVAPI32.dll	97
Cabinet.dll	4
CRYPT32.dll	26
cryptdll.dll	6
DNSAPI.dll	2
FLTLIB.DLL	2
MPR.dll	2
NETAPI32.dll	9
ODBC32.dll	8
ole32.dll	5
OLEAUT32.dll	4
RPCRT4.dll	41
SHLWAPI.dll	6
SAMLIB.dll	22
Secur32.dll	12
SHELL32.dll	1
USER32.dll	19
USERENV.dll	2
VERSION.dll	3
HID.DLL	7
SETUPAPI.dll	4
WinSCard.dll	11
WINSTA.dll	6
WLDAP32.dll	34
advapi32.dll	3
msasn1.dll	8
ntdll.dll	59
netapi32.dll	3
KERNEL32.dll	141
msvcrt.dll	55

ADVAPI32
BCRYPT
CERTCLI
CRYPT32
DBGHELP
DNSAPI
IMAGEHLP
IPHLAPI
KERNEL32
Kernel32
MPR
MSVCRT
NETAPI32
NTDLL
OLE32
OLEAUT32
PSAPI
RPCRT4
SECUR32
SHLWAPI
USER32
VERSION
WLDAP32
WS2_32
WSOCK32
WTSAPI32

IMPLANT FINGERPRINTING SURFACE

- There are multiple techniques in total. The PoC tool we're shipping walks through all of them

Network_public

Network_private

Network_tunneling

Dynamic_code

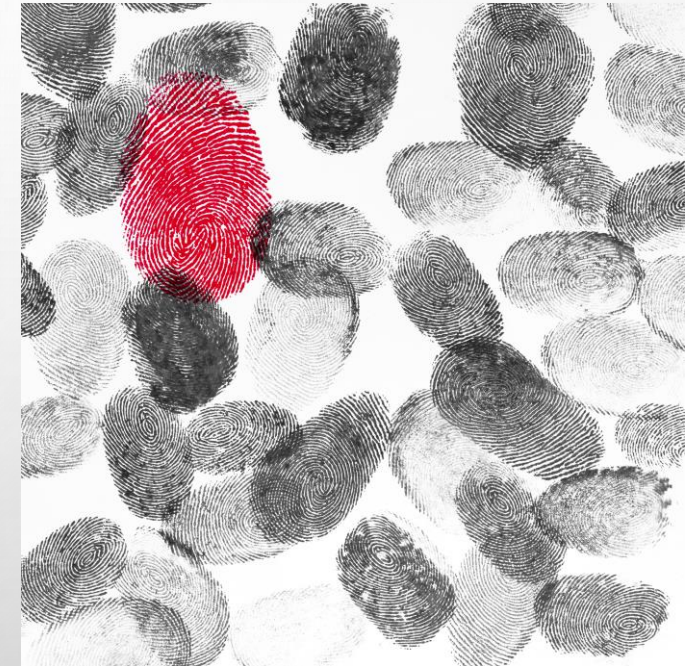
Dynamic_code_nested

Sleep_tp_callback

Proxy_tp_callback

Module_ballooning

High_num_of_common_service_port



HOW WE SCORE AN IMPLANT

- *Tag: a single observation. One technique emits a tag. Each carries a weight*
- *Aggregate: aggregate logically related tags; tags mutually reinforce one another*
- *Severity-score: the aggregated weight of all tags. Higher = stronger confidence*

The more tags combine, the higher the score. Each tag carries a weight and aggregating them produces a severity score - our confidence that we're looking at an implant.

Base Signals (No Escalation)

COMBINATION	LEVEL	RATIONALE
<code>dynamic_code</code> alone	None	Common in JIT, .NET — not suspicious by itself
<code>dynamic_code_nested</code> alone	None	Amplifier only — requires base context
<code>net_public</code> + <code>dynamic_code</code>	Low	Unbacked code with outbound connectivity
<code>net_private</code> + <code>dynamic_code</code>	Low	Unbacked code with internal connectivity
<code>module_balloon</code> + <code>dynamic_code</code>	Low	Offensive DLL accumulation + unbacked execution
<code>module_balloon</code> + <code>net_private</code>	Low	Offensive DLLs + internal reach
<code>module_balloon</code> + <code>net_public</code>	Low	Offensive DLLs + outbound reach
<code>net_public</code> + <code>net_private</code>	Low	Dual-homed / tunneling indicator

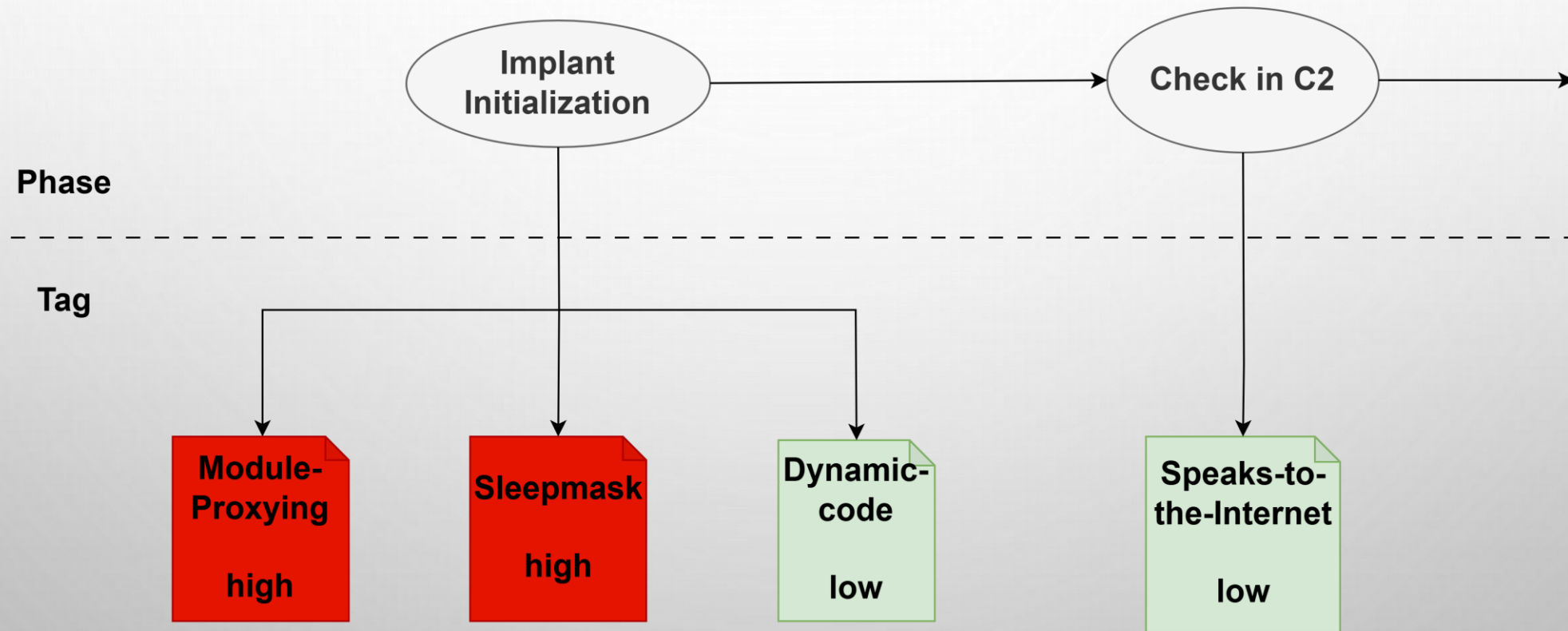
Medium Signals (Three-way Combinations)

COMBINATION	LEVEL	RATIONALE
<code>net_public</code> + <code>dynamic_code</code> + <code>net_tunnel</code>	Medium	Unbacked code tunneling traffic
<code>net_private</code> + <code>dynamic_code</code> + <code>net_tunnel</code>	Medium	Unbacked code with internal tunnel
<code>module_balloon</code> + <code>dynamic_code</code> + <code>net_public</code>	Medium	Offensive DLLs + unbacked code + outbound
<code>module_balloon</code> + <code>net_private</code> + <code>net_public</code>	Medium	Offensive DLLs + dual connectivity
<code>high_svc_ports</code> + <code>dynamic_code</code>	Medium	Port scanning/lateral movement + unbacked code
<code>high_svc_ports</code> + <code>module_balloon</code>	Medium	Port scanning + offensive DLL accumulation
<code>high_svc_ports</code> + <code>net_tunnel</code>	Medium	Port scanning + tunneling

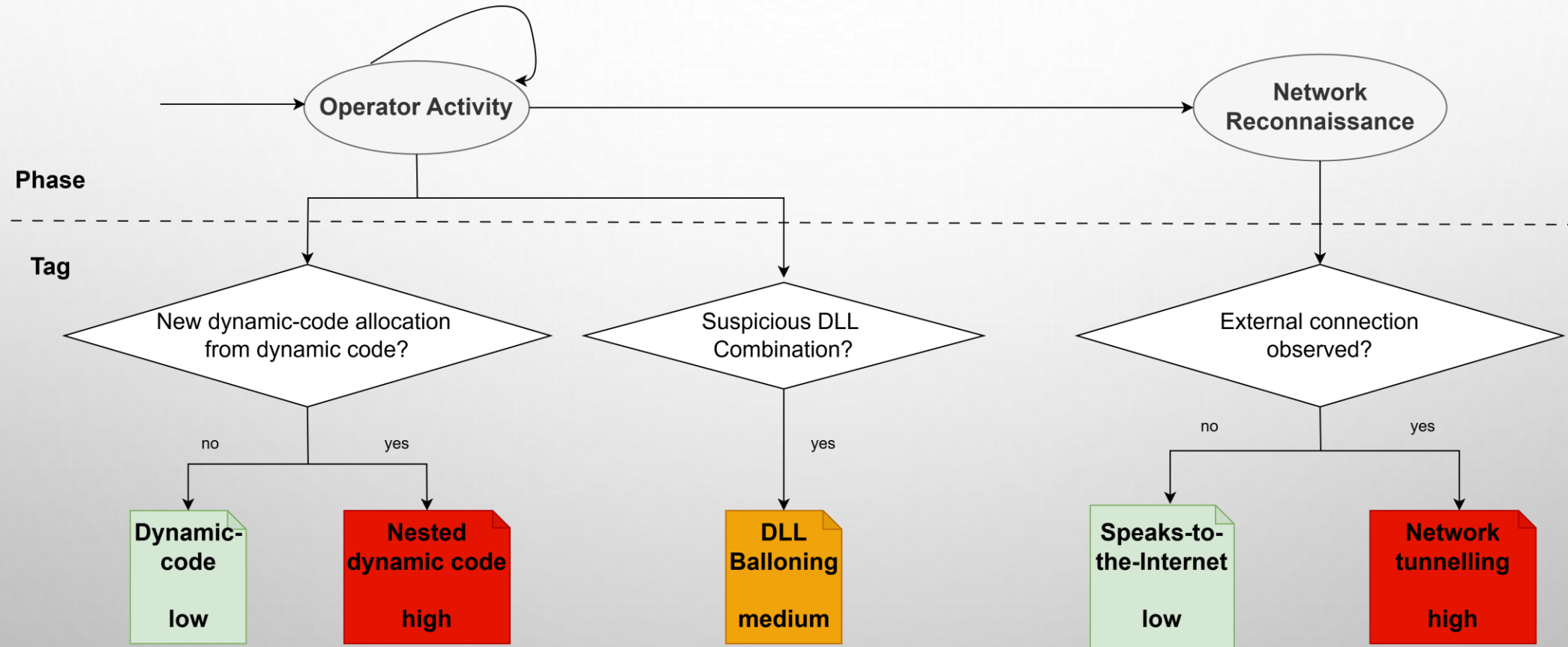
High Signals (Four-way Combinations)

COMBINATION	LEVEL	RATIONALE
<code>net_public</code> + <code>dynamic_code</code> + <code>net_tunnel</code> + <code>module_balloon</code>	High	Full implant pattern — outbound
<code>net_private</code> + <code>dynamic_code</code> + <code>net_tunnel</code> + <code>module_balloon</code>	High	Full implant pattern — internal

TAGGING IMPLANT BEHAVIOR STAGE BY STAGE (1)



TAGGING IMPLANT BEHAVIOR STAGE BY STAGE (2)



DEMO



IT'S A BACK AND FORTH

- The current approach to detect (nested) dynamic code relies heavily on broken callstacks
 - The moment an attacker adds unwind infos to all dynamic locations the detection breaks
 - RtlAddFunctionTable 🤔
- Asynchronous execution of code can be achieved without timers (i.e. Sleepmasks/module proxying)
- Process-Splitting

The story continues ...



GENERAL SUMMARY

- Some scanner capabilities of tools like Moneta and HSB can be implemented based on runtime telemetry
- This enables a shift away from a purely trigger-based detection model
- We can continuously observe characteristic implant behavior as it executes
- Performance overhead is a factor to be considered, but it is manageable
- Implant fingerprinting:
 - *False positives can be reduced by aggregating multiple signals (i.e., tags), combining implant core primitives and evasion telemetry per process and correlating them over time to form an implant fingerprint rather than relying on single indicators*
 - *The more signals (tags) we implement, the more precise the implant fingerprint becomes, making it increasingly difficult to evade the successful fingerprint*



QUESTIONS



END

DOMINIK PHILLIPS

 @dphillips__

SEBASTIAN FELDMANN

 @thefLinkk

https://github.com/threathunters-io/tracebit_x33fcon_2026